

GOLANG DEVELOPER ASSESSMENT

TOY BLOCKCHAIN AND LEDGER SIMULATOR

Name : Theekshana Tishen Alahakoon

Email : tishen.a@botcalm.com

Repo Link : [Repo ↗](#)

Introduction

Before this assessment, my knowledge of blockchain was limited. I had some practical cryptocurrency trading experience, but I did not understand how a blockchain actually works behind the scenes.

I had already started learning Go, so I completed Sangam Mukherjee's "[Golang Full Course](#)." This helped me transition from a MERN stack mindset to working with Go concepts like structs, slices, and pointer receivers. Because of this, my main challenge was not the programming syntax. The real challenge was building a solid understanding of blockchain mechanics from scratch.

The most helpful and clear resources I found were:

- 3Blue1Brown's "[But how does bitcoin actually work?](#)"
- Anders Brownworth's "[Blockchain 101 - A Visual Demo](#)" (Parts 1 & 2)
- Anders Brownworth's [interactive site](#)

While these gave me a solid basic understanding, I used AI tools like Gemini to clarify deeper concepts and answer specific questions.

This report is an engineering log of that process. It covers the design decisions, testing observations, and practical challenges of building a toy blockchain from the ground up.

Investigations

1. Tamper Evidence

To test the tamper-evidence of the blockchain, I deliberately altered a transaction in an existing block.

A terminal window with a dark background. The prompt 'Blockchain>' is in blue, followed by the command 'validate' in white. The output 'Chain is valid!' is shown in green on the next line.

Figure 1: Initial chain validation confirming all blocks are valid before the tamper test.

First, I ran the validation command to ensure the initial chain was fully valid. I targeted a specific transaction inside Block 3 where the sender "bob" transferred an amount of 5 to the recipient "theek". I manually modified the transaction data by changing the amount from 5 to 50.

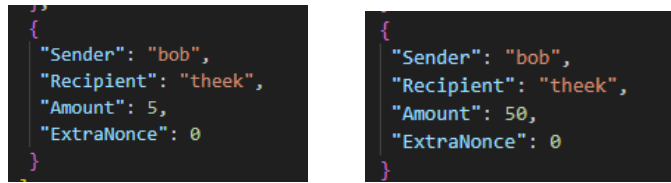


Figure 2: Transaction data in Block 3 before (left, Amount: 5) and after (right, Amount: 50) manually altering the transferred amount.

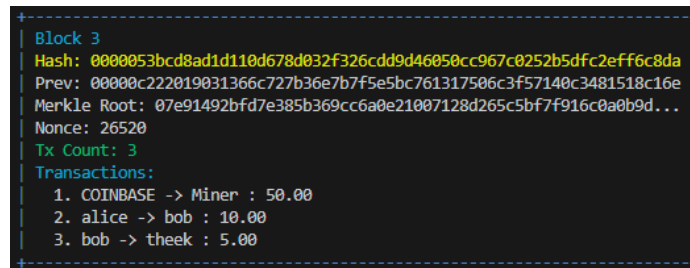


Figure 3a: Block 3 as displayed in the CLI before tampering, showing the original transaction (bob → theek: 5.00).

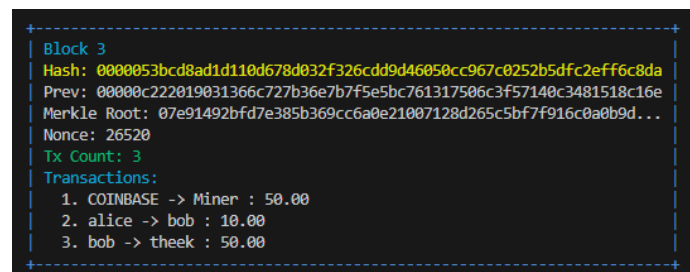


Figure 3b: Block 3 as displayed in the CLI after tampering, with the transaction amount manually changed to 50.00.

After saving the manipulated data, I ran the validation check again. The system successfully detected the tampering and rejected the chain. The console returned a specific error reading "Chain is INVALID. Failed at Block 3: Merkle Root mismatch".

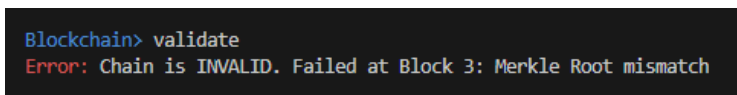


Figure 4: Validation after tampering, showing the chain correctly rejected due to a Merkle root mismatch at Block 3.

2. Difficulty versus effort

To understand how mining difficulty impacts performance, I ran an experiment to mine blocks at several different difficulty levels. During this process, I recorded the time taken and the total number of hashes tried for each level to see how effort scaled.

```
C:\go run mining_experiment.goop\bcsm>go run min
```

Difficulty	Time Taken	Hashes Tried
1	570.9µs	7
2	0s	152
3	581.7µs	152
4	54.6735ms	72162
5	2.3554899s	3166522
6	25.1886639s	24717312
7	1m25.5091376s	78711764
8	16m26.336255s	1013714240

Figure 5: Mining time and hash attempts recorded across difficulty levels 1–8, showing exponential growth in effort as difficulty increases.

Based on these numbers, the trend is clearly not linear. The effort required grows exponentially as the difficulty increases.

At lower difficulties, the blocks are mined almost instantly. However, the time and computational effort scale up rapidly. For example, difficulty 5 took about 2.3 seconds and required over 3 million hashes. Increasing the difficulty to 8 required over 1 billion hashes and took more than 16 minutes to solve.

This exponential growth happens because of how cryptographic hashing works. Adding a requirement for one more leading zero to a valid block hash drastically reduces the total number of acceptable hashes. Because valid hashes become much rarer, the system must generate and test a significantly larger number of random nonces to find a successful match.

3. Design write-up

My first version hashed transactions directly into the block hash. I built a string by concatenating each transaction's sender, recipient, and amount, then combined that with the block's height, timestamp, previous hash, and nonce, and ran the whole thing through a single SHA-256.

This worked, but it tied the block header to the raw transaction list. Hashing the block meant re-serializing every transaction every time, and there was no clean way to prove a single transaction belonged to the block without exposing the whole list.

Once I added the Merkle root, I moved to a better design. Transactions are now hashed independently and combined pairwise up into a single Merkle root, and only that fixed-length value goes into the header. I also switched from a single SHA-256 to a double SHA-256 (hashing twice), which is the same approach Bitcoin uses and gives some protection against length-extension attacks.

The final hash is built from:

```
fmt.Sprintf("%s%s%d%d%d", PrevHash, MerkleRoot, Timestamp, Difficulty, Nonce)
```

Here's why each field is included, in this order:

- PrevHash : this is what actually chains the blocks together. It's the hash of the previous block, so if anyone changes an old block, its hash changes, which breaks the PrevHash pointer in the next block, and the chain falls apart from that point onward.
- MerkleRoot : a fixed-length summary of every transaction in the block. Changing even one transaction (sender, recipient, or amount) changes the Merkle root, which changes the block hash. This replaced my old approach of hashing the raw transaction list directly.
- Timestamp : records when the block was created and keeps the chain in chronological order.
- Difficulty : locked into the hash so nobody can later claim they mined at a harder difficulty than they actually did.
- Nonce : the value miners increment during proof-of-work until the hash meets the difficulty target.

Discussion

1.How does the previous-hash link make tampering with an old block impractical in a real chain, even though it is trivial in your local toy?

In my toy, tampering is trivial because the chain only exists on my own machine. Nobody else has a copy to check against, so nothing stops me from also re-mining every block after the one I changed, since I control all the mining power and there's no one to outrun.

In a real chain, the previous-hash link means changing an old block breaks every hash after it, so I'd have to redo the proof-of-work for that block and all the blocks that follow. Nodes always accept the longest valid chain as the true one, so my tampered version only gets accepted if it ends up longer than the honest chain everyone else keeps extending in the meantime. Since the honest network has far more combined mining power, it grows faster and my version falls behind.

The only way around this is a 51% attack, where an attacker controls more than half the network's mining power and can out-mine everyone else to force their chain to become the longest. This is possible in theory, but on a large network it requires enormous hardware and electricity, so it's economically impractical rather than truly impossible. So it's the previous-hash link plus proof-of-work plus the longest chain rule together that make tampering impractical in a real chain.

2. Proof-of-work is one way to decide who may add the next block. Name at least one alternative, such as proof-of-stake or proof-of-authority, and give one advantage and one drawback versus proof-of-work in your own words.

One alternative is proof-of-stake. Instead of miners competing by burning computational effort, validators are chosen to add the next block based on how much of the native currency they've locked up (staked). The more you stake, the higher your chance of being selected.

Advantage: proof-of-stake is far more energy efficient. There's no arms race of hardware running constant hash computations, so it avoids the massive electricity cost that comes with proof-of-work mining.

Drawback: it can favor wealth concentration. Since your chance of validating is tied to how much you already hold, participants with large stakes keep earning more block rewards, which can make it harder for smaller holders to meaningfully participate over time. Proof-of-work, in contrast, only requires access to computing power, which is a more open barrier to entry.

Sources : [Proof of Work vs Proof of Stake: What's Better?](#)

3. List three concrete ways your toy differs from a production blockchain (think about consensus among peers, transaction signatures, Merkle trees, and finality). Pick one and sketch how you would add it.

- **No peer consensus or finality.**

My chain runs as a single local process. There's no network of nodes agreeing on which chain is valid, so there's no real trust model, just one copy of the truth that I control. This also means there's no concept of a block being "final" in the sense of surviving a fork, since finality in a real network comes from the rest of the network building on top of a block, and that doesn't apply when there's only one node.

- **No transaction signatures.**

My transactions just record a sender, recipient, and amount as plain data. Anyone could construct a transaction claiming to be from any sender, since there's no cryptographic proof that the sender authorized it. This is also why I could get away with a FAUCET command to mint test coins out of thin air (`addtx FAUCET Alice 100`), which isn't how real Bitcoin faucets work at all, a real faucet is just someone with existing coins giving some away, not a protocol feature. I only used it to skip mining dozens of blocks before I could test transactions.

- **ExtraNonce as its own field instead of reused space.**

In real Bitcoin, the Coinbase transaction has no signature to verify since it isn't spending from anyone, so miners store the extra nonce inside the empty scriptSig space that would normally hold a signature. My toy doesn't have a signature field at all yet, so there was no existing space to reuse, and I had to add ExtraNonce as a separate typed field instead. Once

I add real signatures, ExtraNonce could move into that same unused signature space on the Coinbase transaction, matching how Bitcoin does it, instead of needing its own field.